7.1

```
// Use Euclid's algorithm to calculate the GCD.
private long GCD(long a, long b)
{
  // Repeat until we're done
  while (true)
  {
    // Set remainder to the remainder of a / b
    long remainder = a % b;

    // If remainder is 0, we're done. Return b.
    if (remainder == 0) return b;

    // Set a = b and b = remainder.
    a = b;
    b = remainder;
  }
}
```

7.2

When the code changes but comments are not updated so the comments become incorrect and the comments are written before the code was fully understood making it correct or vague

7.4

To prevent errors i could check for invalid inputs like both a and b being 0, throw an exception or handling edge cases explicitly, and adding validation before running the algorithm

```
if (a == 0 && b == 0)
{
  throw new ArgumentException("GCD is undefined for a = 0 and b = 0.");
}
```

7.5

Yes because it prevents crashes from invalid inputs, makes the program more reliable, and helps users understand what went wrong. Without error handling, edge cases could cause undefined behavior.

7.7

1. Get into the car
2. Start the car
3. Exit current location
4. Drive toward the main road
5. Follow roads toward the nearest supermarket
6. Enter the supermarket parking lot
7. Park the car

Assuming the driver knows how to drive, the car works and has gas, the driver knows where the supermarket is or has GPS, and roads are accessible

8.1

```
def gcd(a, b):
  a = abs(a)
  b = abs(b)

  if a == 0:
     return b
  if b == 0:
     return a

  while True:
     remainder = a % b
     if remainder == 0:
        return b
     a = b
     b = remainder


def isRelativelyPrime(a, b):
  if a == 0:
     return b == 1 or b == -1
  if b == 0:
     return a == 1 or a == -1

  return abs(gcd(a, b)) == 1


# Test
test_values = [(-10, 10), (8, 9), (21, 35), (0, 1), (0, 2), (17, 19)]
```

```
for a, b in test_values:
  print(a, b, isRelativelyPrime(a, b))
```

8.3

I used black box and white box testing. I could use black box when validating functionality, white box when debugging logic and gray box when you partially understand the internals

8.5

```
def areRelativelyPrime(a, b):
  if a == 0:
     return b == 1 or b == -1
  if b == 0:
     return a == 1 or a == -1

  return abs(gcd(a, b)) == 1
```

Bugs i found were forgetting absolute value in GCD → incorrect results for negatives and missing edge cases for 0

Testing helped me catch edge cases (0, negatives), confirmed correctness across multiple inputs, and increased confidence in the function

8.9

Black box testing because it tests all possible inputs and verifies outputs without needing to understand the internal implementation.

8.11

Estimated total=A×BoverlapEstimated total=overlapA×B

Alice = {1,2,3,4,5} → 5 bugs

Bob = {2,5,6,7} → 4 bugs

Overlap (Alice ∩ Bob) = {2,5} → 2 bugs

Estimate=5×42=10Estimate=25×4=10

10 bugs found; union = {1,2,3,4,5,6,7,8,9,10}

$10-10=010-10=0$

0 bugs remain

8.12

the denominator becomes 0 so the estimate becomes undefined or extremely large. This means the testers are likely not overlapping enough and the estimate becomes unreliable. The lower bound estimate is the number of bugs